CORE103L Computational Thinking
LW6 - Strings



Department of Computer Science

Dhanani School of Science and Engineering

Habib University

Fall 2026

Copyright @ 2026 Habib University

Contents

1 Introduction **2**
1.1 Instructions 2
1.2 Marking scheme 2
1.2.1 Lab Objectives 2
1.3 Late submission policy 2
1.4 Use of AI 3
1.5 Viva 3

2 Prerequisites **4**
2.1 Assert Statements 4
2.2 Pytest 4

3 Lab Exercises **6**
3.1 Campus Challenge Code 6
3.2 Campus Display Rotation 7
3.3 Debugging: Student Portal Password Checker 9
3.4 Campus Cleanup Crew 10
3.5 Group Chat Message Expander 11

Introduction

1.1 Instructions

- The deadline for submission of this lab is at the end of lab time.
- The lab must be submitted online via CANVAS. You are required to submit a zip file that contains all the *.py* files.
- The zip file should be named as *LW6_aa123456.zip* where *aa123456* will be replaced with your student id.
- **Files that don't follow the appropriate naming convention will not be graded.**
- **Your final grade will comprise both your submission and your lab performance.**

1.2 Marking scheme

This lab will be marked out of 10.

1.2.1 Lab Objectives

The following are the lab objectives of this lab:

- Iterate over and manipulate strings in Python.
- Use string indexing and slicing where appropriate.
- Build and transform strings based on specified rules.
- Choose suitable approaches for solving string-processing problems.
- Parse structured information contained within strings.
- Debug programs involving string processing and iteration.

1.3 Late submission policy

There is no late submission policy for the lab.

1.4 Use of AI

Taking help from any AI-based tools such as ChatGPT is strictly prohibited and will be considered plagiarism.

1.5 Viva

Course staff may call any student for Viva to provide an explanation for their submission.

Prerequisites

2.1 Assert Statements

Assert statements are crucial tools in programming used to verify assumptions and test conditions within code. For instance, in Python, an assert statement is typically as follows:

```
assert condition, message
```

For example, in a function calculating the square root, in order to ensure that the input value is non-negative, one might add the following assert statement:

```
assert x >= 0, 'x should be a non-negative number'
```

If the condition evaluates to False, an AssertionError is raised, highlighting the issue and its corresponding message.

2.2 Pytest

Pytest is a popular testing framework in Python known for its simplicity and power in facilitating efficient and scalable testing for applications and software. In order to use Pytest, one first needs to install it and create test functions prefixed with *test_* in a Python file. To install Pytest, you can write the following command on the terminal:

```
pip install pytest
```

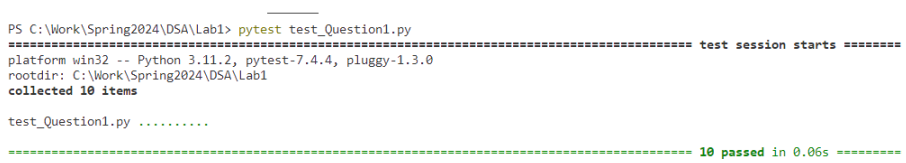
After installing Pytest, one can execute all the tests by typing the following command on the terminal:

```
pytest
```

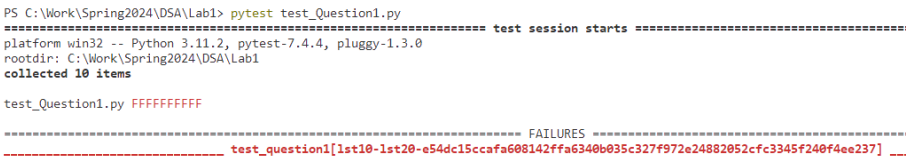
If you want to test any particular file, then you will have to specify the filename as follows:

```
pytest <filename>.py
```

If all the tests pass successfully, then you should receive a similar output.



However, if one or more tests fail, you should receive a similar output.



Lab Exercises

3.1 Campus Challenge Code

This question should be implemented in the file ***q1.py****.*

During a campus challenge, each team receives an event message. To obtain the event code, the team keeps every character whose position is a multiple of `3` or `5`, counting positions from `1`. If a position is a multiple of both, its character is included only once.

Given an event_message, generate the event code while preserving the original order of the selected characters.

Note: Positions in this question are counted starting from `1`. Therefore, position `1` corresponds to Python index `0`, position `2` corresponds to index `1`, and so on. A position that is a multiple of both `3` and `5` is included only once.

Constraints

- `isinstance(event_message, str)` is `True`.
- Use **iteration**.

Function Specification

Write an **iterative** function named **event_code** that takes one parameter, **event_message**, and returns the event code.

Sample Interaction

```
>>> event_code('12345678901234567890123456789O')
'35690258014570'

>>> event_code('pomegranate')
'mgrat'
```

Sample Explanation

For the first example, the positions that are multiples of `3` or `5` are:

```
Positions:  3  5  6  9  10  12  15  18  20  21  24  25  27  30
Characters: 3  5  6  9   0   2   5   8   0   1   4   5   7   0
```

For example, positions `15` and `30` are multiples of both `3` and `5`, but their characters are included only once.

Reading the selected characters from left to right gives `'35690258014570'`.

For `'pomegranate'`, the selected positions are:

```
Positions:  3  5  6  9  10
Characters: m  g  r  a   t
```

Therefore, the event code is `'mgrat'`.

Testing

To test your function, type:

```
pytest tests/test_q1.py
```

3.2 Campus Display Rotation

This question should be implemented in the file ***`q2.py`***.

During a university event, a message is shown on a looping digital display. A positive rotation moves the first character to the end, while a negative rotation moves the last character to the beginning.

Rotations are cyclic. For `'campus'`, which has length `6`, rotating by `8` is the same as rotating by `2`, and rotating by `-8` is the same as rotating by `-2`. A rotation count of `0` leaves the message unchanged.

Constraints

- `isinstance(display_message, str)` is `True`.
- `isinstance(rotation_count, int)` is `True`.
- $1 \leq$ `len(display_message)` ≤ 1000

Function Specification

Write a function named **`rotate_message`** that takes **`display_message`** and **`rotation_count`**, and returns the rotated message.

Sample Interaction

```
>>> rotate_message('campus', 3)
'puscam'

>>> rotate_message('campus', -2)
'uscamp'

>>> rotate_message('campus', 8)
'mpusca'

>>> rotate_message('campus', -8)
'uscamp'
```

Sample Explanation

For `rotate_message('campus', 3)`, move the first three characters to the end:

```
cam | pus  ->  pus | cam  ->  puscam
```

For `rotate_message('campus', -2)`, move the last two characters to the beginning:

```
camp | us  ->  us | camp  ->  uscamp
```

For `8` rotations, `6` rotations complete one full cycle, so only `2` more positive rotations are needed:

```
campus -> ampusc -> mpusca
```

For `-8` rotations, `6` negative rotations complete one full cycle, so only `2` more negative rotations are needed:

```
campus -> scampu -> uscamp
```

Hint: Using String Slicing

```
display_message = 'campus'

>>> display_message[:2]
'ca'

>>> display_message[2:]
'mpus'
```

Testing

To test your function, type:

```
pytest tests/test_q2.py
```

3.3 Debugging: Student Portal Password Checker

This question should be implemented in the file `q3.py`.

The university is testing a password checker for student portal accounts, but the current program is producing incorrect results. Your task is to debug the provided implementation.

A password is **accepted by the student portal** if:

1. its length is greater than `8` and less than `30`;
2. it contains at least one uppercase letter;
3. it contains at least one lowercase letter;
4. it contains at least one digit;
5. it contains no spaces.

The conditions must be checked in the order shown above. The program should print the message for the **first** condition that is not satisfied. If all conditions are satisfied, it should print the required success message `'Strong Password!'`.

The file `q3.py` already contains a buggy implementation of **check_password_strength** and its helper functions. Debug the provided code so that the password checker works correctly.

Constraints

- `isinstance(password, str)` is `True`.
- Do not change the names or parameters of the provided functions.
- Do not remove any of the provided helper functions.
- Fix and use the provided helper functions in **check_password_strength**.

Required Messages

Your function must print the appropriate message exactly as shown below:

```
Length must be between 8 and 30 characters (exclusive).
There must be at least one upper case character.
There must be at least one lower case character.
There must be at least one digit.
There must be no spaces in the password.
Strong Password!
```

Sample Interaction

```
>>> check_password_strength('sd2j42n 35')
There must be at least one upper case character.

>>> check_password_strength('ASO3QSA')
Length must be between 8 and 30 characters (exclusive).

>>> check_password_strength('StrongPass9')
Strong Password!
```

In the first sample, the password also contains a space, but the missing uppercase letter is reported because that condition is checked first.

Testing

To test your function, type:

```
pytest tests/test_q3.py
```

3.4 Campus Cleanup Crew

This question should be implemented in the file ***q4.py****.*

A campus-cleanup simulation represents a walkway using:

- `'L'` – litter
- `'V'` – a volunteer
- `'E'` – an empty spot

Volunteers are processed from **left to right**. Each volunteer picks up at most one piece of litter:

1. If there is litter immediately to the right, it is removed and replaced with `'E'`.
2. Otherwise, if there is litter immediately to the left, it is removed and replaced with `'E'`.
3. Otherwise, nothing happens.

Changes to the walkway take effect immediately before the next volunteer is processed.

Constraints

- `isinstance(walkway, str)` is `True`.
- $1 \leq$ `len(walkway)` ≤ 1000
- walkway contains only `'L'`, `'V'`, and `'E'`.
- Use **iteration**.

Function Specification

Write an **iterative** function named **clean_walkway** that takes one parameter, **walkway**, and returns the final state of the walkway after all volunteers have acted.

Sample Interaction

```
>>> clean_walkway('LVLEL')
'LVEEL'

>>> clean_walkway('VLLVE')
'VEEVE'
```

Sample Explanation

For `'LVLEL'`, the volunteer has litter immediately to the right, so it is removed:

```
L V L E L
    |
    v
L V E E L
```

Therefore, the result is `'LVEEL'`.

For `'VLLVE'`, volunteers act from left to right:

```
V L L V E
V E L V E   <- first volunteer removes litter on the right
V E E V E   <- second volunteer removes litter on the left
```

Therefore, the result is `'VEEVE'`.

Testing

To test your function, type:

```
pytest tests/test_q4.py
```

3.5 Group Chat Message Expander

This question should be implemented in the file ***q5.py****.*

A student group-chat bot supports a shortcut for repetitive text. Before a message is posted, the bot expands any section written in the form `k[text]`, where k is the repeat count.

For example:

```
3[ha]
```

represents:

```
hahaha
```

A message may contain several encoded sections as well as ordinary characters.

Given an encoded_message, return the message that the bot would post after expanding all encoded sections.

Constraints

- `isinstance(encoded_message, str)` is `True`.
- $1 \leq$ `len(encoded_message)` ≤ 1000
- Every repeat count k satisfies $1 \leq k \leq 50$.
- A repeat count may contain more than one digit.
- Digits occur only as repetition counts in encoded sections.
- The characters `[` and `]` occur only as delimiters of encoded sections.
- Text inside brackets contains at least one character and does not contain digits or additional brackets.
- Every encoded section is correctly written in the form `k[text]`.
- Use **iteration**.

Function Specification

Write an **iterative** function named **expand_message** that takes one parameter, **encoded_message**, and returns the expanded message.

Each encoded section `k[text]` must be replaced by `text` repeated k times. Characters outside encoded sections must remain in their original order.

Sample Interaction

```
>>> expand_message('3[ha]')
'hahaha'

>>> expand_message('2[go] team!')
'gogo team!'

>>> expand_message('2[abc]3[cd]ef')
'abcabccdcdcdef'

>>> expand_message('12[!] done')
'!!!!!!!!!!!! done'
```

Sample Explanation

For `'3[ha]'`, the text inside the brackets is `'ha'` and the repeat count is `3`:

```
3[ha]  ->  ha + ha + ha  ->  hahaha
```

For `'2[go] team!'`, only `'go'` is repeated:

```
2[go] team!  ->  go + go + " team!"  ->  gogo team!
```

For `'2[abc]3[cd]ef'`, each encoded section is expanded separately:

```
2[abc]  ->  abcabc
3[cd]   ->  cdcdcd

abcabc + cdcdcd + ef
-> abcabccdcdcdef
```

For `'12[!] done'`, the character `'!'` is repeated `12` times, followed by the remaining text:

```
12[!] done
-> !!!!!!!!!!!! done
```

Testing

To test your function, type:

```
pytest tests/test_q5.py
```